

What Counts as the Prompt?

Seven Failed Definitions and a Unit-of-Analysis Problem

Watson Hartsoe

Working paper · 18 August 2026



Working Paper · AI-augmented research process
Evidence, prompts, and making history preserved.

Abstract

This paper does not offer a new definition of the prompt. It argues that the repeated failure of such definitions exposes a prior methodological problem. A prompt can be treated as a command, description, program, specification, context-bearing utterance, production trace, or component of a distributed system. Each description catches a real mechanism. Each also fails when evidence from another part of the same field is allowed to press on it. Deictic expressions can act without containing their own reference; descriptions become consequential only inside apparatuses; programming knowledge exceeds program text; specifications continue to change through repair and failure; “context” collapses world models, discourse history, live coupling, and situated position; provenance does not decide authorship; and distribution does not by itself identify transformation, authority, or responsibility. The wager advanced here is therefore methodological rather than ontological: research on prompting has a unit-of-analysis problem. Before an effect is attributed to “the prompt,” the analysis must state what has been included in that object, what has been excluded, and what surrounding state has been held fixed. This wager does not settle what a prompt is. It makes the instability of the object itself testable.

Keywords: prompting; natural-language programming; unit of analysis; situated action; specification; authorship; distributed cognition; human-computer interaction

1 A bad beginning: define the prompt

A familiar scholarly move is to begin by defining the object. Here that move keeps failing.

Call the prompt a command and a phrase such as “that one” becomes awkward: the operative reference may have been built across prior interaction rather than encoded in the command itself. Call it a description and the sentence becomes too powerful: the words do not act alone; an engineered apparatus makes some descriptions consequential. Call it a program and older programming research intrudes from both directions: natural language was executable long before language models, while programming itself was never exhausted by program text. Call it a specification and failure, repair, demonstration, and tacit judgment keep adding material after the supposed specification has already been written. Call all of that “context” and several unlike mechanisms disappear into one convenient word. Call the whole process distributed and another problem appears: distribution names multiplicity without yet telling us which transformations, permissions, judgments, and responsibilities matter.

The temptation is to solve this embarrassment with a larger noun. “Distributed prompt program,” “closure topology,” “operational language,” “worldtext”: each can become a container large enough to stop the argument from leaking. That is exactly the danger. A concept can appear strong because it has absorbed every counterexample.

The wager of this paper is smaller and harder to evade: *before asking what a prompt is, prompt research must declare its unit of analysis*. If an effect is attributed to a visible string while the causal work is being done by interaction history, tool descriptions, interface selection, a world model, model parameters, human evaluation, or external state, then “the prompt caused X” is not yet an explanation. It is a boundary choice that has not been defended.

This is not a claim that there is one correct boundary. The evidence below points in the opposite direction. Different questions require different units. The problem begins when one unit is named and another is silently used.

2 Seven definitions under pressure

Candidate 1: The prompt is a command

The command picture is not foolish. Contemporary agent systems do distinguish between language that requests analysis and language that authorizes action; tool-enabled systems can cross from discourse into external state change. Austin also gives a much older reason not to reduce language to description: utterances can participate in actions when procedures and circumstances make those acts felicitous (Austin 1962).

But command is already too narrow for ordinary prompt practice. The working manuscript notices how much can be done with expressions such as “this,” “that one,” “again,” and “like before.” Suchman makes the deeper point that intelligibility can depend on particulars of the occasion rather than on a self-sufficient expression (Suchman 1985). Clark and Wilkes-Gibbs similarly show reference as an interactional accomplishment in which candidate expressions are repaired, expanded, replaced, or accepted (Clark and Wilkes-Gibbs 1986).

The failure is precise. If the research unit is only the current imperative sentence, the analysis can miss the work by which its referents and action relevance became available. Yet enlarging the unit to “the conversation” does not automatically fix this. Selection state, visual scene, tool state, and prior actions may matter too. The command picture therefore leaves an unanswered measurement question: *which prior state must be included before the command becomes the same command?*

Candidate 2: The prompt is a description

“Describe the room and the room appears” is an irresistible story because it makes contemporary generation feel like a change in what description itself can do. The working manuscript distrusts that sentence for good reason. A description does not become an operation by semantic promotion. Someone builds a machine in which particular descriptions have effects.

Winograd makes the missing machinery unusually visible. In SHRDLU, English is interpreted against a designed blocks world containing represented entities, relations, procedures, and state; interpreted sentences can then become executable instructions (Winograd 1973). Austin supplies a different but compatible warning: the force of language depends on procedures and circumstances, not only on sentence form (Austin 1962).

The description picture fails when it attributes to the words what belongs to the arrangement around them. But the corrective can fail too. Saying “the apparatus does it” risks making language irrelevant. The open question is not which side wins. It is how to partition contribution without pretending the partition is given in advance.

Candidate 3: The prompt is a program

There are strong reasons to use programming vocabulary. Current systems can synthesize temporary code from natural-language instructions, constrain output with schemas, and route permitted tools through generated control flow. Earlier systems make the genealogy harder, not weaker. SHRDLU already mapped English into procedures; Attempto deliberately restricted English so specifications

could enter formal and executable representations (Winograd 1973; Fuchs and Schwitter 1996); Pygmalion made concrete demonstration part of program construction decades before prompt iteration (Smith 1993).

Yet “prompt = program” breaks against Naur from inside programming itself. Naur argues that programming competence includes a theory of the matters handled by the program that exceeds program text and documentation (Naur 1985). Program text is formal without being the whole of programming practice. The analogy therefore cannot be: code is complete, prompts are incomplete. That contrast mistakes one artifact for an activity.

There is still a real difference: programming languages provide formal syntax and execution semantics that ordinary-language prompts generally do not. Naur does not erase this. The point is narrower. “Program” does not settle the unit. It relocates the same question: does the relevant program consist of the user’s words, generated code, tool schemas, runtime state, tests, or the operator’s theory of how those pieces behave together?

Candidate 4: The prompt is a specification

Specification looks safer because it need not imply direct executability. A first prompt can be treated as requirements rather than finished program text; failures can produce new constraints; examples can narrow acceptable outcomes; tests can make some acceptance conditions executable. This vocabulary captures much of contemporary iterative practice.

Then the specification refuses to stay put. Suchman’s analysis of repair treats trouble as an occasion for diagnosing competing interpretations rather than merely reissuing a fixed plan (Suchman 1985). Pygmalion shows concrete examples becoming specification material through interaction (Smith 1993). The working manuscript adds a more difficult case: an operator may reliably reject an output before being able to state the criterion that governs the rejection. The criterion is operative without yet being a proposition.

This yields two bad simplifications. If the prompt is “the specification,” then every revision threatens to make the object retrospectively infinite. If the specification is only what can be stated before execution, tacit judgment and failure-driven learning are excluded by definition. Neither answer should be rescued by calling everything a “living specification.” That phrase may only move the boundary problem into a more flattering noun.

Candidate 5: The prompt is language plus context

“Context” is where many of the previous failures go to hide.

Winograd’s architecture distinguishes utterance, discourse, world state, and procedural knowledge rather than treating context as one undifferentiated supplement (Winograd 1973). Suchman’s situation particulars are not simply extra text appended to a message (Suchman 1985). Brooks pushes harder: a system may need continued coupling to a changing world rather than a richer representation of that world (Brooks 1991). The working manuscript raises another asymmetry: a human can occupy a position, history, and bodily situation whose practical role may change when converted into explicit context.

So “add context” can mean at least four different interventions: add more symbols, expose more interaction history, provide a represented world state, or let the system re-perceive and act in a changing environment. Those mechanisms should not be made equivalent by vocabulary.

The difficult question is experimental. If the same visible prompt is held fixed and only live world access changes, has the prompt changed? A study can answer yes or no by stipulation. What it cannot do is switch answers halfway through the causal explanation.

Candidate 6: The prompt is the production trace

Provenance seems to offer an escape from unstable definitions. Keep everything: prompts, generations, revisions, rejects, selections, transformations. Then the process can be inspected rather than guessed.

The working manuscript breaks the stronger hope. A complete trace can show what happened without deciding what category should be assigned to what happened. The same trace can support rival authorship judgments if the evaluators privilege different operations: origination, specification, selection, revision, endorsement, responsibility, or sentence production. Provenance is evidence for an attribution rule; it is not the attribution rule itself.

This matters beyond authorship. A trace can accumulate without identifying which events were causally necessary, which were incidental, and which old instructions remained behaviorally active after revision. More evidence does not automatically produce a better unit of analysis. Sometimes it only gives a larger pile from which the boundary must still be chosen.

Candidate 7: The prompt is part of a distributed system

Distribution is the most tempting final refuge because it can accommodate nearly everything the smaller definitions exclude. Hutchins provides a rigorous reason to take distributed organization seriously: cognition can be analyzed through transformations and propagation of representational state across people and artifacts (Hutchins 1995).

But “distributed” earns explanatory force only when the transformations are named. Human-model writing can involve generating, prompting, selecting, rejecting, revising, arranging, evaluating, authorizing, and publishing. Calling the whole chain distributed does not tell us which operation produced a sentence, which operation made it part of the work, or which operation made someone answerable for it. The working manuscript’s authorship fragments are useful here precisely because they do not settle the matter: causal production can be distributed while responsibility remains concentrated.

Distribution therefore does not rescue the object. It replaces “where is the prompt?” with several more exact questions: what representation moved, who or what transformed it, what authority was exercised, which state changed, and which consequences attached to the change?

3 What survives the wreckage

One claim survives these collisions without requiring a new ontology: the unit of analysis must be declared rather than smuggled in.

Suppose a researcher writes an utterance U and observes an outcome Y . Between them may stand interaction history H , system and developer instructions S , model/runtime configuration M , tool contracts T , represented or live environment state E , and human selection or revision J . The notation

$$Y = F(U, H, S, M, T, E, J, \dots)$$

is not a theory of prompting. It is bookkeeping. Its only purpose is to make a causal sentence harder to utter carelessly. If a study says that a change in U caused a change in Y , what was held fixed? If “the prompt” includes H in one experiment and excludes it in another, are the results commensurable? If J is necessary for the final artifact but absent from the declared unit, what exactly is being attributed to the prompt?

This is where the earlier thesis of “local closure” fails as a master explanation. Closure can be a useful description of some transitions: a schema makes one output shape admissible; a confirmation step gates a mutation; a repair settles a reference for the next move. But it does not subsume situatedness, tacit evaluation, authorship, or the theory a programmer carries beyond text. Treating closure as the architecture beneath all of them merely replaces one unstable object with another.

A better methodological wager is more modest: *claims about prompts should identify the analysis boundary that makes the claim true*. Sometimes the useful unit will be the visible string. Sometimes the interaction. Sometimes the tool-equipped agent. Sometimes the human-machine production process. The choice is not innocent, and it is not permanent.

4 A reporting discipline, not a definition

The practical consequence is not a taxonomy of what prompts “really are.” It is a discipline for describing experiments and artifacts.

When an effect is attributed to a prompt, report at least the following boundaries when they matter:

- **Utterance:** the exact user-visible words or multimodal input under test.
- **History:** prior turns, demonstrations, corrections, and persistent memories available at the moment of interpretation.
- **Instruction apparatus:** system/developer instructions, tool descriptions, schemas, policies, and permission surfaces not authored in the focal utterance.
- **World access:** represented state, selected objects, files, sensors, APIs, or possibilities for renewed perception.
- **Execution:** generated procedures, tool calls, intermediate state changes, and commit gates.
- **Human aftermath:** rejection, selection, revision, arrangement, endorsement, and publication.

This list is intentionally heterogeneous. It should not be promoted into a universal architecture. Its value is diagnostic: it makes visible which pieces a particular claim has omitted.

The same discipline also changes how failures are read. A bad output may reveal a hidden interpretation, as repair-oriented accounts suggest, or it may be stochastic noise. A successful output may indicate robust grounding, or it may hide a lucky misunderstanding. Without identifying the unit and the changing variables, neither success nor failure is self-interpreting evidence.

5 Questions that should remain open

The field does not yet support several answers that would make the paper cleaner.

First, there is no demonstrated minimal unit that is always sufficient. A visible string may be enough for one controlled task and absurdly incomplete for another.

Second, “situation” cannot simply be renamed “context.” Suchman and Brooks point to different reasons: occasion-specific intelligibility and ongoing environmental coupling are not obviously recoverable by adding more text (Suchman 1985; Brooks 1991).

Third, tacit criteria remain a problem for reproducibility. The fact that an operator can repeatedly say “no” before articulating why does not prove an ineffable faculty; it also does not justify pretending the criterion has already been formalized.

Fourth, provenance does not settle authorship or responsibility. A production trace can constrain an argument, but the attribution rule must come from somewhere else. More scholarship is required before claims about authorship can be made confidently.

Fifth, historical novelty remains unresolved. Winograd, Pygmalion, Attempto, Naur, Suchman, Agre and Chapman, Brooks, and current agent systems do not form a neat genealogy. Similarity is not influence. Older mechanisms may reappear at new scales without making the current systems merely old ideas with larger models. The comparison has to be done operation by operation.

These unresolved points are not defects to be concealed before publication. They are the places where the object still moves when pressed.

6 Conclusion: do not let the noun settle the dispute

The prompt may be a useful practical noun and a bad theoretical object. That is not yet a conclusion about what prompts are. It is a research problem produced by repeated boundary failures.

Command captures authorization but misses interactional reference. Description captures wording but misses apparatus. Program captures execution but not the theory and machinery around program text. Specification captures constraint but not the ways failure, examples, and tacit judgment continue to change what counts as the task. Context gathers surroundings by collapsing unlike mechanisms. Provenance preserves history without supplying a verdict. Distribution recognizes multiplicity while leaving transformation and accountability to be specified.

The next move should therefore not be another sovereign definition. It should be an experiment that makes the cut visible. Hold some components fixed. Vary others. State which object is being called “the prompt.” Watch where the explanation breaks.

If one stable unit eventually survives those tests, that would be a finding. At present, the more defensible claim is that we have been arguing about an object whose boundary changes with the question.

References

- Austin, J. L. (1962). *How to Do Things with Words*. Ed. by J. O. Urmson. Oxford: Clarendon Press.
- Brooks, Rodney A. (1991). “Intelligence without Representation”. In: *Artificial Intelligence* 47.1–3, pp. 139–159.
- Clark, Herbert H. and Deanna Wilkes-Gibbs (1986). “Referring as a Collaborative Process”. In: *Cognition* 22.1, pp. 1–39. DOI: 10.1016/0010-0277(86)90010-7.
- Fuchs, Norbert E. and Rolf Schwitter (1996). *Attempto Controlled English (ACE)*. arXiv: [cmp-lg/9603003](https://arxiv.org/abs/cmp-lg/9603003). URL: <https://arxiv.org/abs/cmp-lg/9603003>.
- Hutchins, Edwin (1995). *Cognition in the Wild*. Cambridge, MA: MIT Press.
- Naur, Peter (1985). “Programming as Theory Building”. In: *Microprocessing and Microprogramming* 15, pp. 253–261.
- Smith, David Canfield (1993). “Pygmalion: An Executable Electronic Blackboard”. In: *Watch What I Do: Programming by Demonstration*. Ed. by Allen Cypher. MIT Press. URL: <https://acypher.com/wwid/Chapters/01Pygmalion.html>.
- Suchman, Lucy A. (Feb. 1985). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Tech. rep. ISL-6. Palo Alto, CA: Xerox Palo Alto Research Center.
- Winograd, Terry (1973). “A Procedural Model of Language Understanding”. In: *Computer Models of Thought and Language*. Ed. by Roger C. Schank and Kenneth M. Colby. San Francisco: W. H. Freeman, pp. 152–186.

Appendix A: Assembly instrument

This working paper belongs to a portable research field. The exact assembly instrument is preserved as `_PROMPTS/10__slipcase-v15.55-AM__assembly.poml`. Assembly prompt SHA-256: `a5ef0e2e3ea56463e25065f19e7313b4e9c8c569b8cceb1a3c5d9f05bd83598a`.

Appendix B: Making history

The prior field’s 40 immutable zettel payloads were preserved without rewriting. The previous compiler wager – “the program is the topology of closure boundaries” – was not deleted or retroactively corrected; it was moved into prior-resource history and marked as a failed governing interpretation. The user correction against artificial fluency and the subsequent “so cook” command are preserved verbatim in `_PROMPTS/`. No new external web research was introduced in this compile. Full making history is preserved in `what-counts-as-the-prompt__MAKING_HISTORY.txt`.

Appendix C: Replication path

The companion `index.html` is a standalone replication capsule containing all 40 exact zettel payloads, graph state, bibliography, consequential prompts, the current paper, source map, making history, and rebuild instructions. See `000__RETURN_PATH.txt` for checkpoint identity and rejoin phrase.